Creating a Stock Prediction App in Python

Ajk Follow 6 min read · Apr 18, 2024

Developed by Ayush Kulkarni

This project is a culmination of both Flask, a Python Framework, along with Matplotlib, Sklearn, and Yahoo Finance. In this project I decided to utilize a Linear Regression Model from sklearn in order to predict future trends. I specifically chose the Linear Regression Model so that I could efficiently predict the future stock prices of given stocks. Combining this with a dataset taken from Yahoo Finance, I was able to create a better dataset and model. Yahoo Finance gave me an up-to-date dataset, which I was able to implement as the training set and test set data for my Linear Regression model. This dataset is compromised of features such as, “Close Price”, “Open Price”, “Volume”, “High”, and “Low”, however, I did need to add on another feature called “Date” so that I could keep track of the days. Here’s a look into a little bit of the raw data from one of the companies, Google.

```
         Date        Open        High         Low       Close   Adj Close    Volume
0  2023-04-13  106.470001  108.264999  106.440002  108.190002  108.190002  21650700
1  2023-04-14  107.690002  109.580002  107.589996  109.459999  109.459999  20758700
2  2023-04-17  105.430000  106.709999  105.320000  106.419998  106.419998  29043400
3  2023-04-18  107.000000  107.050003  104.779999  105.120003  105.120003  17641400
4  2023-04-19  104.214996  105.724998  103.800003  105.019997  105.019997  16732000
5  2023-04-20  104.650002  106.888000  104.639999  105.900002  105.900002  22515300
6  2023-04-21  106.089996  106.639999  105.485001  105.910004  105.910004  22379000
7  2023-04-24  106.050003  107.320000  105.360001  106.779999  106.779999  21410900
8  2023-04-25  106.610001  107.440002  104.559998  104.610001  104.610001  31408100
9  2023-04-26  105.559998  107.019997  103.269997  104.449997  104.449997  37068200
10 2023-04-27  105.230003  109.150002  104.419998  108.370003  108.370003  38235200
11 2023-04-28  107.800003  108.290001  106.040001  108.220001  108.220001  23957900
12 2023-05-01  107.720001  108.680000  107.500000  107.709999  107.709999  20926300
13 2023-05-02  107.660004  107.730003  104.500000  105.980003  105.980003  20343100
14 2023-05-03  106.220001  108.129997  105.620003  106.120003  106.120003  17116300
Data: (252, 7)
```

Raw Data from Yahoo Finance for GOOG

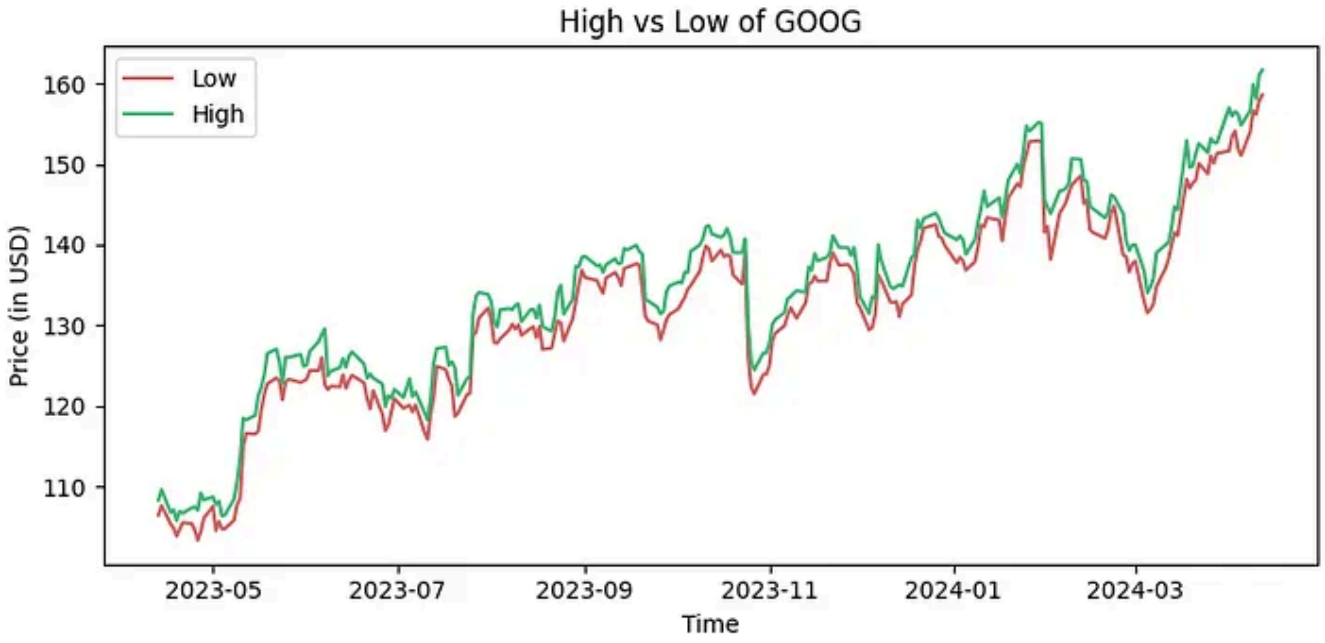
And below is the code I used to produce this. I used the datetime module in Python in order to make the time period for the stock more flexible for users.

```
from datetime import date
import pandas as pd
import yfinance as yf

howmanyyears = int(input("How many years? > ")) # <-- Getting user input for yea
today = date.today()
END_DATE = today.isoformat()
START_DATE = date(today.year - howmanyyears, today.month, today.day).isoformat()

whichstock = input("Which stock? > ") # <-- Getting user input for stock name
data = yf.download(whichstock, start=START_DATE, end=END_DATE)

data.reset_index(inplace=True)
data['Date'] = pd.to_datetime(data.Date) # <-- Inserting the 'Date' Feature

# Outputting the first 15 rows of data
print(data.head(15))
print(f"Data: {data.shape}")
```

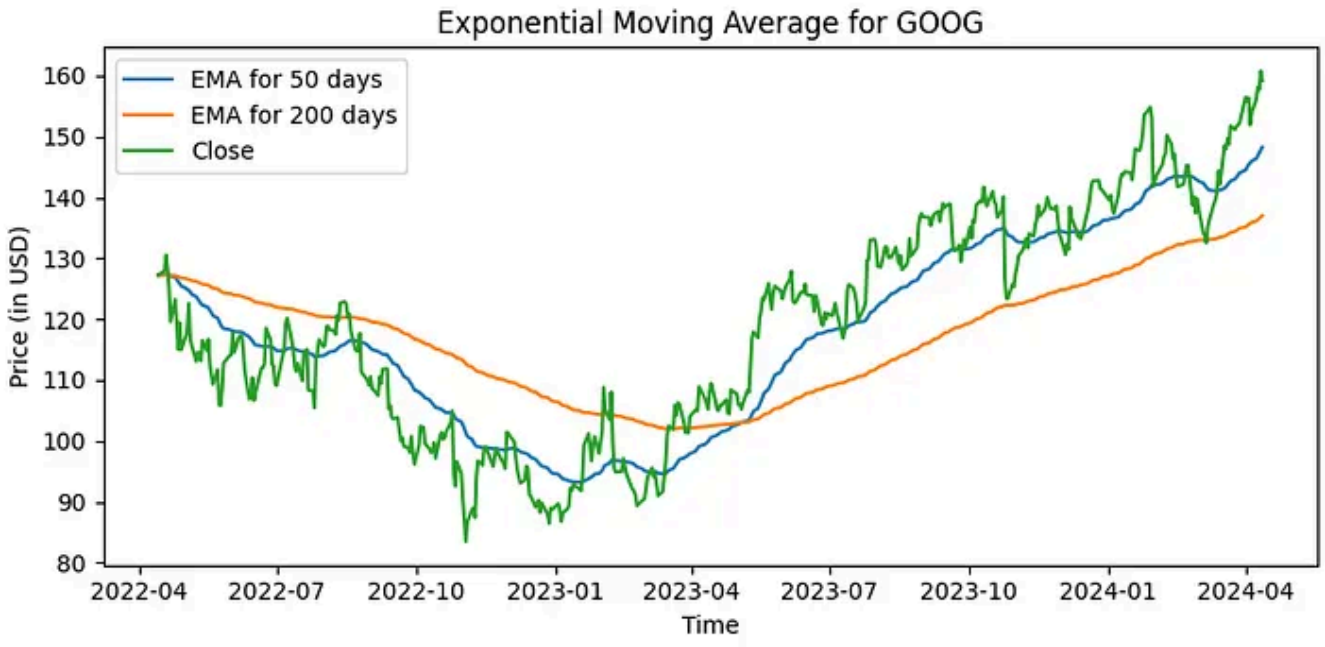
Additionally, I added on two extra features: the Exponential Moving Average for 50 days and 200 days. Through doing this, I was able to predict whether the stock market was tending to be Bearish or Bullish during a certain period of time and show users more trends about their stock.

```
data['EMA-50'] = data['Close'].ewm(span=50, adjust=False).mean()
data['EMA-200'] = data['Close'].ewm(span=200, adjust=False).mean()
```

Before making the Regression Model, I wanted to first visualize some of this data. I plotted the High vs Low prices along with a graph of the daily closing price with the Exponential Moving Average for 50 and 200 days.



GOOG High vs Low plotted using matplotlib



GOOG Exponential Moving Average plotted using matplotlib

Here is the code used to generate this graph.

```
# High vs Low Graph
plt.figure(figsize=(8, 4))
plt.plot(data['Low'], label="Low", color="indianred")
plt.plot(data['High'], label="High", color="mediumseagreen")
plt.ylabel('Price (in USD)')
plt.xlabel("Time")
plt.title(f"High vs Low of {stock_name}")
plt.tight_layout()
plt.legend()

# Exponential Moving Average Graph
plt.figure(figsize=(8, 4))
plt.plot(data['EMA-50'], label="EMA for 50 days")
plt.plot(data['EMA-200'], label="EMA for 200 days")
plt.plot(data['Adj Close'], label="Close")
plt.title(f'Exponential Moving Average for {stock_name}')
plt.ylabel('Price (in USD)')
plt.xlabel("Time")
plt.legend()
plt.tight_layout()
```

After getting a feel of the dataset, it was time to dive into the Linear Regression Model. The end goal for this project was to predict the Closing Price for a given stock, so I made it the X component. That left me with all the other features as Y components.

```
x = data[['Open', 'High', 'Low', 'Volume', 'EMA-50', 'EMA-200']]
y = data['Close']
```

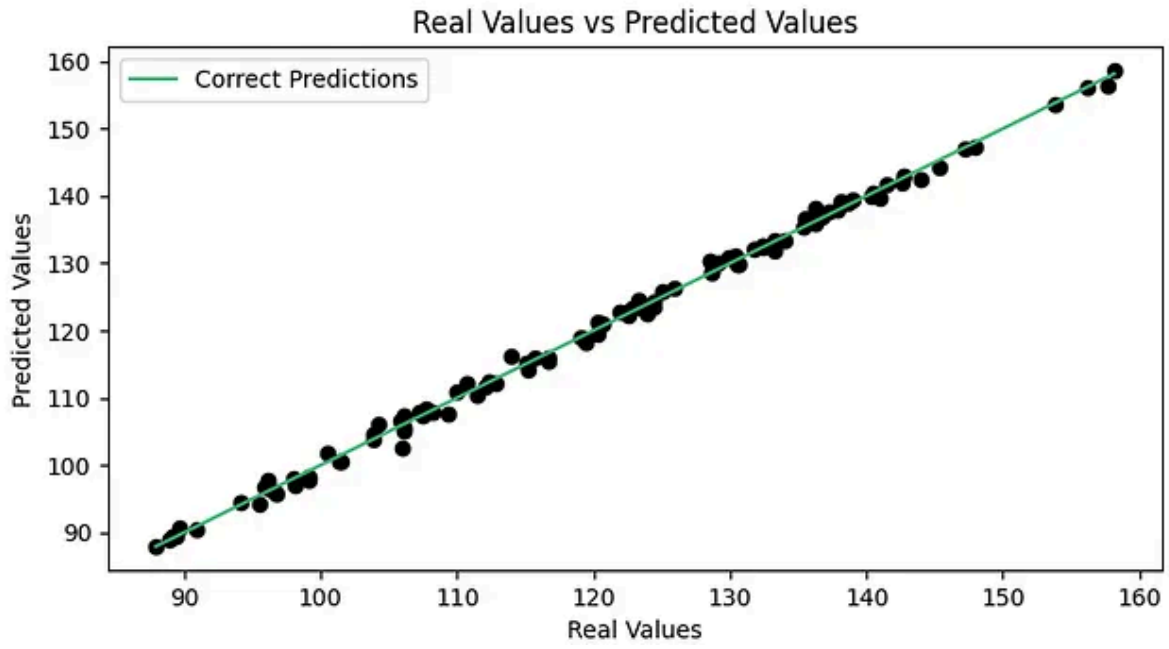
Next, Scikit-learn's `train_test_split` function allowed me to separate the data into 80% for training and the rest 20% for testing.

```
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size = 0.2)
```

Now it was time to fit the Linear Regression Model and predict what the future prices would be.

```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
pred = lr_model.predict(X_test)
```

Once this was done, I wanted to visualize how accurate my model was so I created a graph of the Model's values vs the actual values.



Real vs Predicted Values for Linear Regression Model

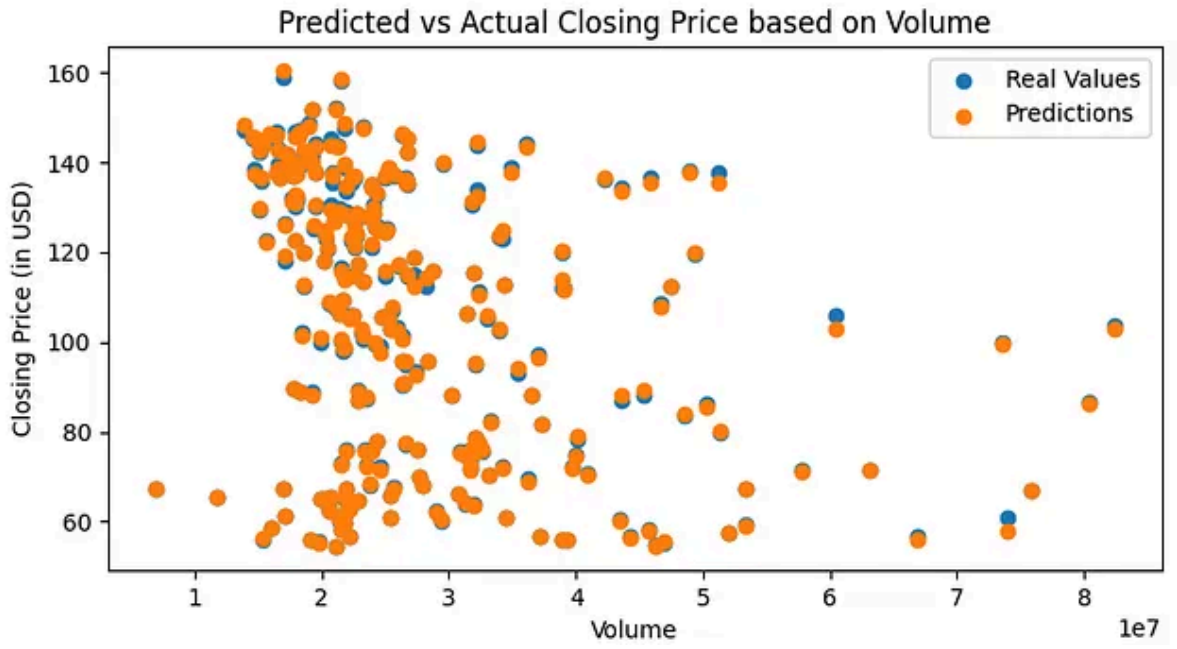
Furthermore, I printed out the Real vs Predicted prices for the stock during random days. This allowed me to explicitly see how close or far apart these numbers were and make it more clear whether my model had worked as intended.

	Actual_Price	Predicted_Price
929	139.660004	141.771002
11	67.073997	67.165848
958	143.539993	141.000513
627	99.570000	100.090610
998	155.869995	154.724970
734	94.250000	94.240259
359	145.205994	143.992757
971	145.320007	144.505774
279	117.254997	117.498794
632	97.180000	98.162604
	Actual_Price	Predicted_Price
count	202.000000	202.000000
mean	114.497755	114.563693
std	24.125463	24.123916
min	60.817001	61.094063
25%	95.342499	95.664102
50%	115.728500	116.227180
75%	136.209629	135.793020
max	159.190002	160.604197

Actual Prices vs Predicted Prices

```
d=pd.DataFrame({'Actual_Price': y_test, 'Predicted_Price': pred})
print(d.head(10))
print(d.describe())
```

After finishing the Model all that was left was to try to predict the closing price based off of the different features. The one that stuck out to me the most was Volume vs Closing Price. I feel that this model worked really well and was able to mostly predict the values with little to no error.

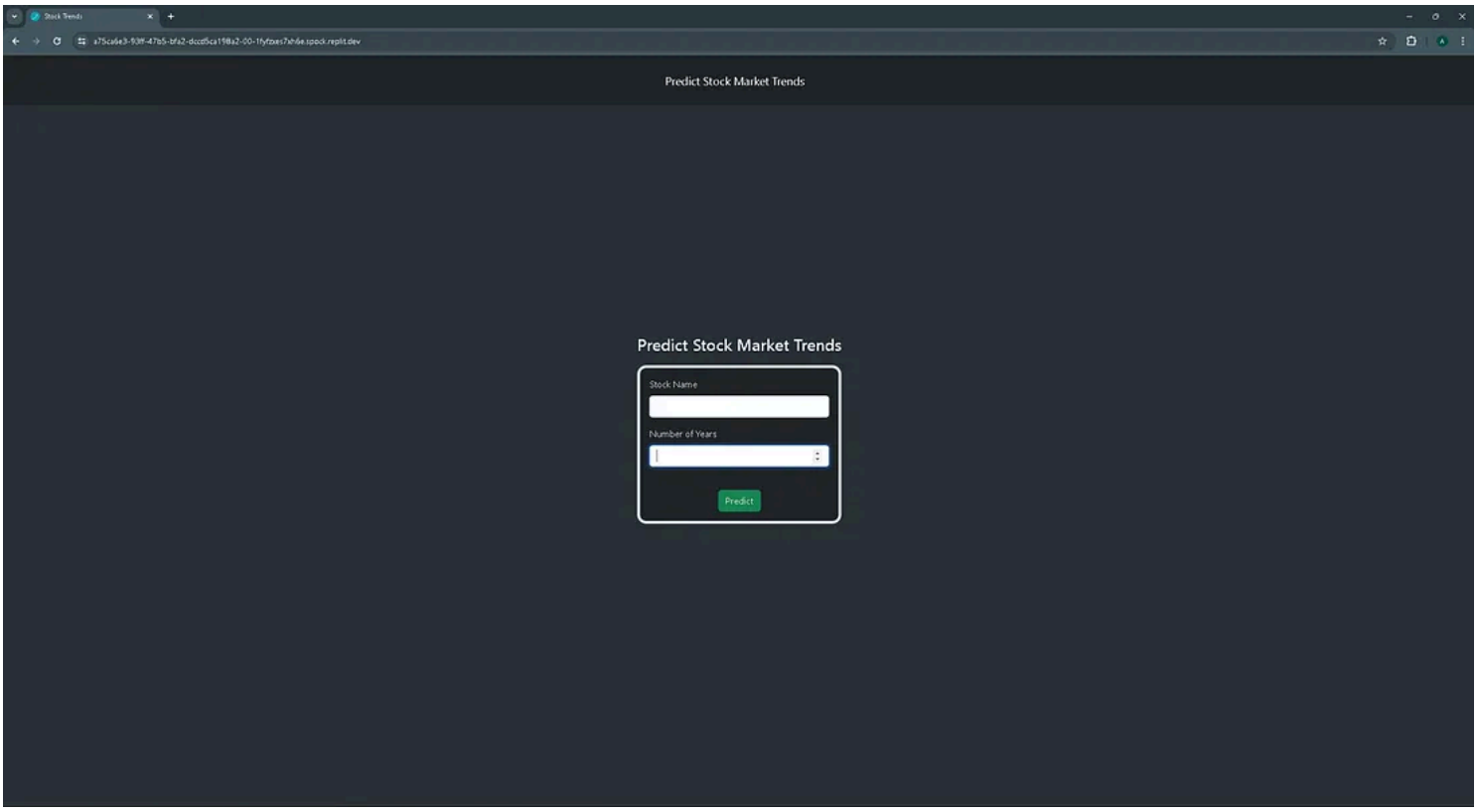


Now time for some statistics. In order to fully see how well my model had performed I had to look at the r^2 score, the mean absolute error, and the mean squared score. Here's a look at each of these values.

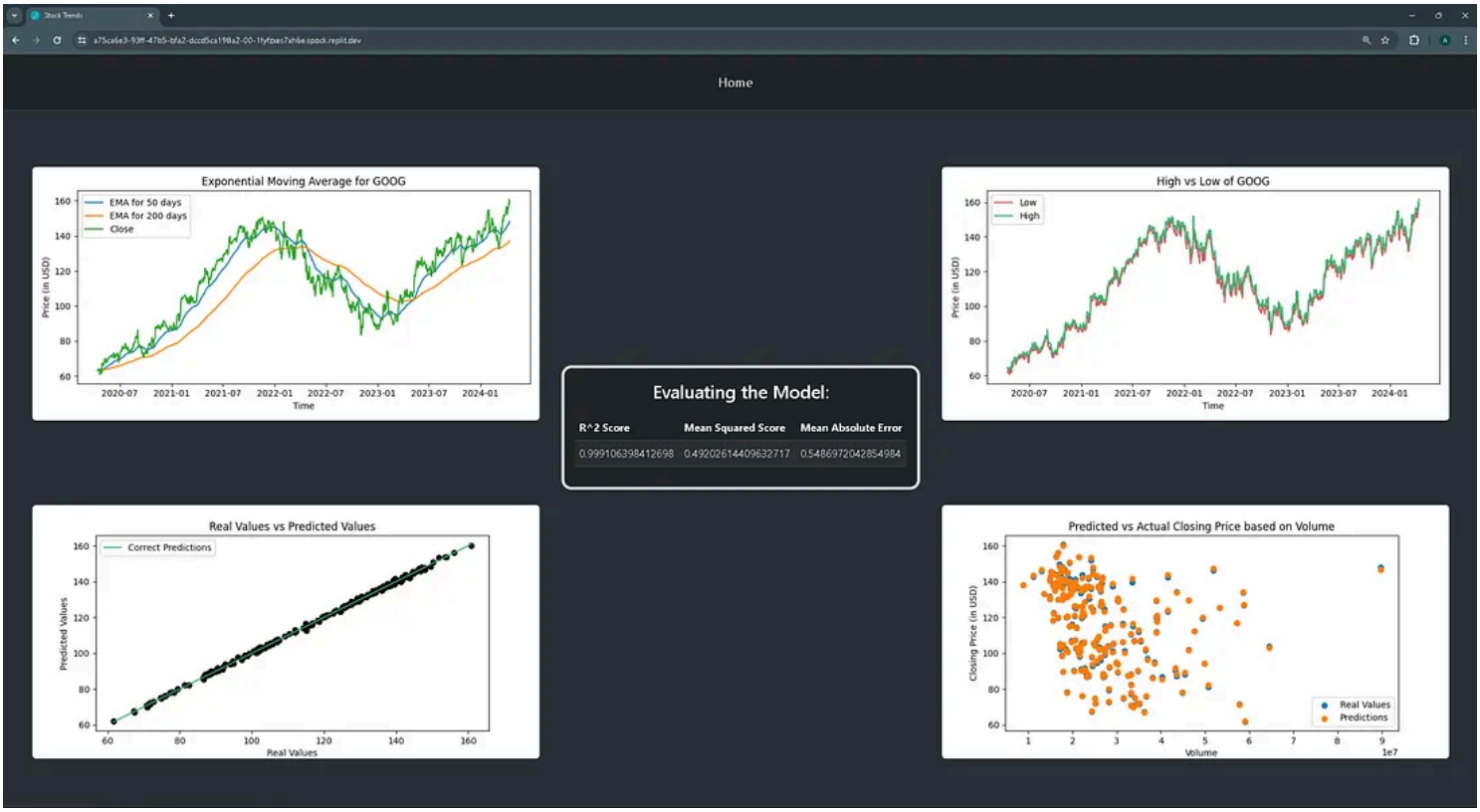
```
Mean Squared Error: 0.5657641347035391
R^2 Score: 0.9990231240723859
Mean Absolue Error: 0.5755113406639947
```

Linear Regression Model Results

My project didn't stop here, however. Previously I had already learned Flask, one of Python's frameworks, so I went on to implement this as an app using that. Additionally, I used Bootstrap for styling and got input from the user using POST Request to the backend. Here is a look at the website, its functionality, and the link if you want to check it out: https://a75ca6e3-93ff-47b5-bfa2-dccd5ca198a2-00-1fyfzxes7xh6e.spock.replit.dev/

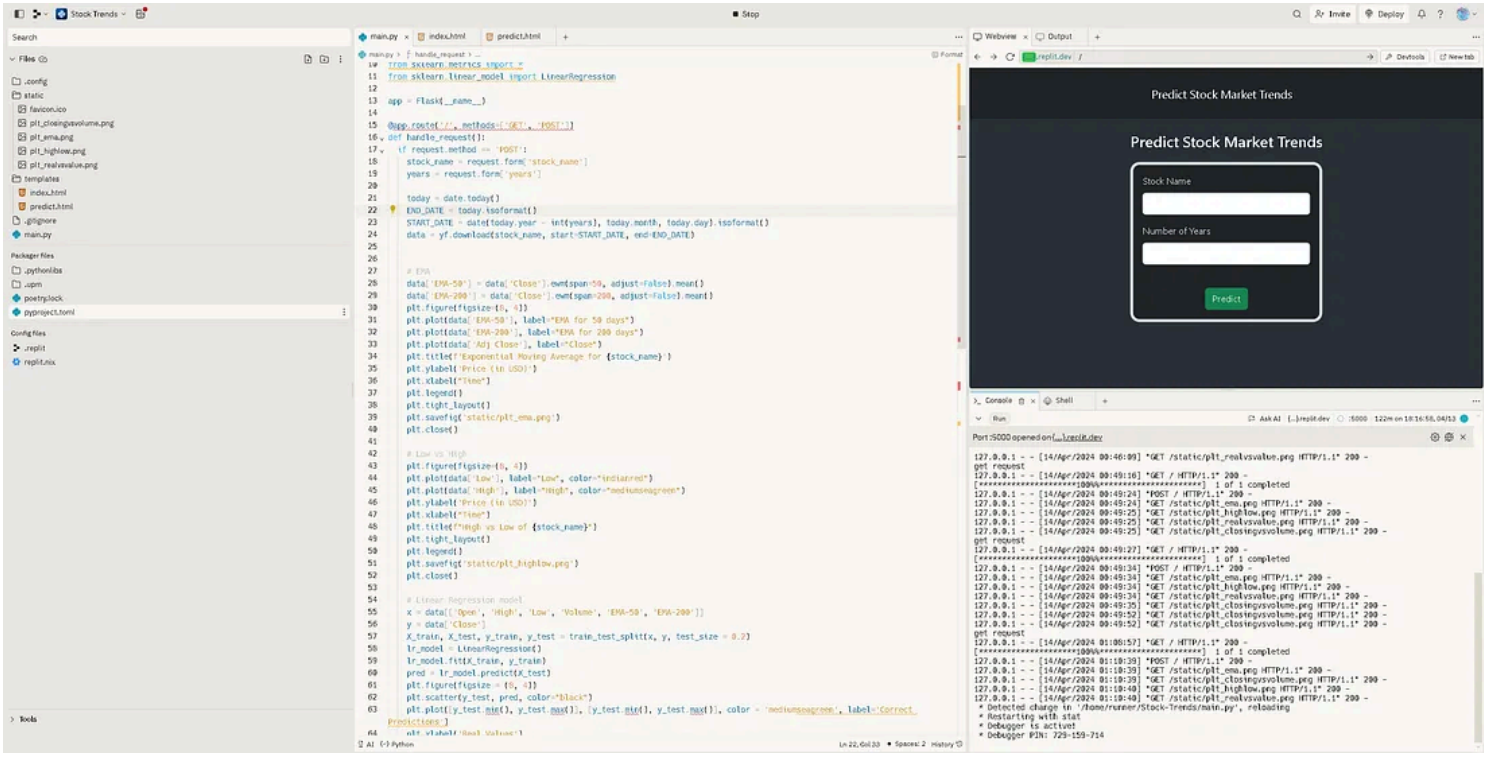


Website Home Page



Website Stock Page

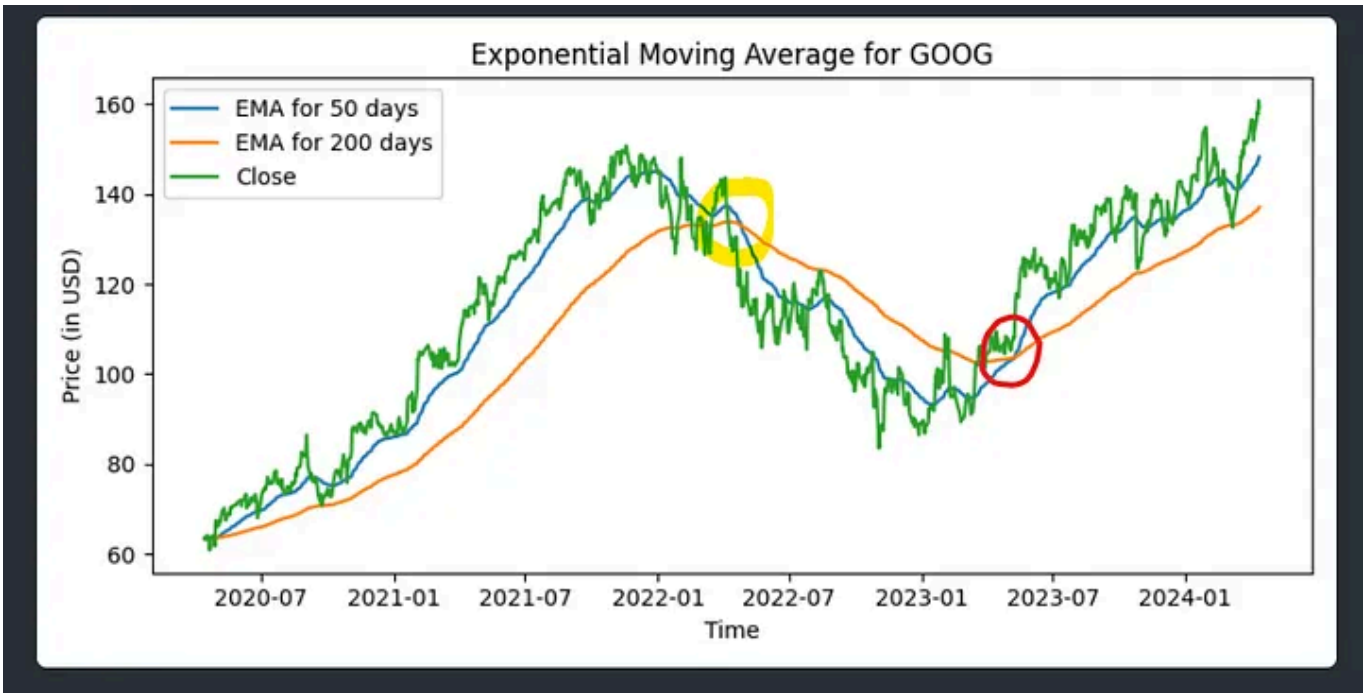
I won't get into the details of the backend, but here is a quick glimpse of the code.



Backend of my Stock Trends App developed on replit.com

Looking back, there are a few changes I would make in my model. One of them being that I would change the Train-Test split from being 80–20 to being only from a certain time range in the past, therefore allowing my model to fully be on its own when predicting the future prices.

Additonally, using a Exponential Moving Average for 50 days and 200 days allowed me to visualize when there would be a Death Cross or Golden Cross. A Death Cross is when a long-term moving average crosses over a short-term moving average. The Golden Cross occurs when a short-term moving average crosses over a major long-term moving average. In yellow is an example of a Death Cross and in red is an example of a Golden Cross found in Google's stock.



Throughout this entire project I was constantly learning new techniques of data analysis and Model creation. Some of these techniques being, refining

datasets to get rid of outliers or null values, creating a Regression Model to predict certain things, and using matplotlib to visualize the graphs and information. Overall, I feel that I'm going to be taking a lot of new skills away from this amazing TRAIN AI course that will very much benefit me in the future.

AI | Python | Matplotlib | Stock Market | Code

Written by Ajk

48 followers · 3 following

JS FullStack Programmer

Follow

Responses (6)

Vamshi Damera

What are your thoughts?

Zach

May 6, 2024

I believe your results look this good, because you are predicting price, not change in price. If a stock is current at 140, it's much more like to be 139 the next minute than 150.

Go back and perform the analysis on how many times your regression... more

2 Reply

Raotap
May 2, 2024

Do you seriously believe that by playing aournd with Python you will be able to predict stock prices? Who are you bullshitting? Yourself or some hapless idiots?

2 Reply

E

elanthamil madhavan
Mar 24, 2025

thanks for the wonderful information and coding details about stock market prediction.

nandri bhaii! ajay anne payalugaaa!

Reply

See all responses

More from Ajk

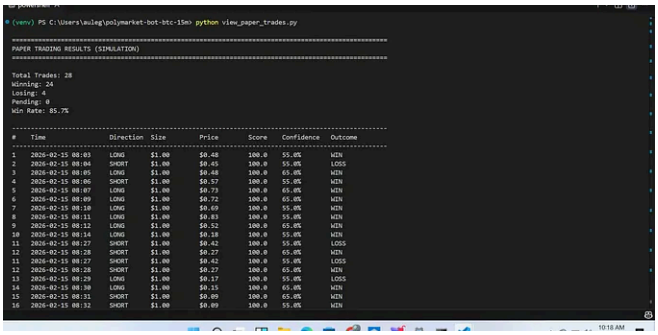
Ajk

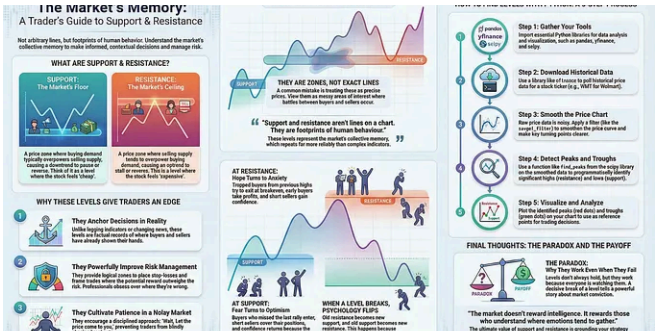
I got Nuxt.js on Repl.it. Here's how you can too!

Dec 1, 2020

See all from Ajk

Recommended from Medium





Aule Gabriel

The Ultimate Guide: Building a Polymarket BTC 15-Minute Tradin...

KEY FEATURES

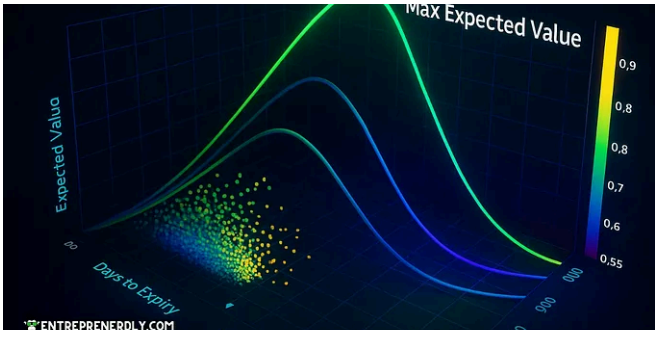
Feb 15 6 1

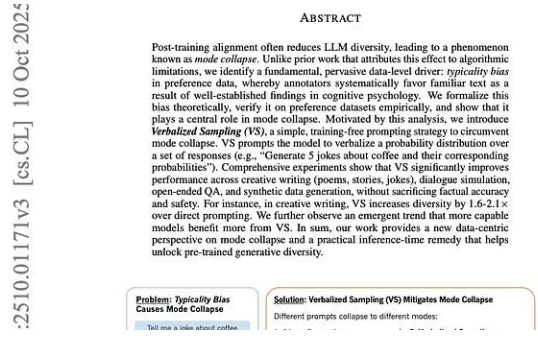
In Python in Plain English by Manish Peshwani

How to find Support and Resistance levels of a stock...

If you've spent enough time staring at stock charts, you'll notice something uncanny.

Jan 10 16





Cristian Velasquez

In Generative AI by Adham Khaled

Scanning Optimal Credit Spreads

Rank OTM put and call spreads by expected value, probability of profit, IV, delta and...

Sep 15, 2025 243 4

Stanford Just Killed Prompt Engineering With 8 Words (And I...

ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...

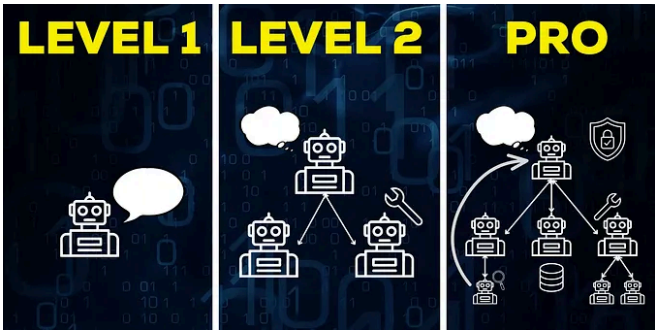
Oct 19, 2025 24K 646

In Artificial Corner by The PyCoach

The Best AI Tools for 2026

If you're going to learn a new AI tool, make sure it's one of these

Dec 1, 2025 4.8K 276



In Data Science Collective by Marina Wyss

AI Agents: Complete Course

From beginner to intermediate to production.

Dec 6, 2025 4K 146

See more recommendations